

APOSTILA COMPLETA

Primeiros Passos em Java

Do JShell ao domínio das bases da linguagem

Hello World • Variáveis • Tipos Primitivos • Operadores • Casting

7 seções • Tipos primitivos • Casting e Conversões

Complete Java Masterclass

Sumário

01	JShell & Hello World	Seu primeiro programa, statements e erros comuns
02	Variáveis e Palavras-chave	Declaração, tipos, camelCase e expressões
03	Tipos Inteiros	int, byte, short, long — range, overflow, underflow
04	Tipos de Ponto Flutuante	float, double, notação científica e sufixos
05	char, boolean e String	Unicode, imutabilidade, concatenação
06	Operadores e Expressões	Aritméticos, %, ++/--, operadores compostos
07	Casting e Conversão de Tipos	Widening, narrowing, cast explícito

CAPÍTULO 01

JShell & Hello World

Seu primeiro programa Java e como explorar a linguagem de forma interativa

O que é o JShell?

O JShell é a ferramenta interativa do Java, disponível desde o JDK 9, que permite executar código Java linha a linha sem a necessidade de criar um projeto completo. É o ambiente perfeito para aprender, experimentar e testar pequenos trechos de código com feedback imediato. Diferente de um IDE, o JShell não exige que você crie classes, métodos main ou compile arquivos — basta digitar uma instrução e ver o resultado na hora.

Para navegar pelo histórico de comandos, use as setas do teclado ↑ e ↓. Para encerrar o JShell, use `/exit` ou `/ex`. Para listar todas as variáveis declaradas na sessão atual, use `/vars`.

Seu Primeiro Programa

A tradição de todo programador é começar com o famoso "Hello World". Em Java, isso é feito com um único statement que utiliza o método `println` da classe `System`, que imprime o texto e pula uma linha ao final.

```
System.out.println("Hello World");

// Saída: Hello World

// println pula linha; print não pula:

System.out.print("Olá ");

System.out.print("Java!");

// Saída: Olá Java! (na mesma linha)
```

O que é um Statement?

Um statement é um comando completo a ser executado pelo computador. Em Java, todo statement termina com ponto e vírgula (`;`) — é a forma que o compilador usa para saber onde uma instrução termina e outra começa. Esquecer o ponto e vírgula é um dos erros mais comuns

para iniciantes.

Java é uma linguagem case-sensitive, o que significa que letras maiúsculas e minúsculas fazem diferença. Por exemplo, System com S maiúsculo é válido; system com s minúsculo causará um erro de compilação.

■ **Dica:** O JShell é um ambiente seguro para experimentar — tudo que der errado pode ser corrigido na próxima linha. Não tenha medo de cometer erros; eles são parte essencial do aprendizado.

CAPÍTULO 02

Variáveis e Palavras-chave

Como declarar variáveis, nomear corretamente e usar expressões

O que é uma Variável?

Uma variável é uma forma de armazenar informação na memória do computador durante a execução do programa. Você define um nome e o Java cuida de reservar o espaço necessário na RAM. Toda variável em Java tem três componentes: um tipo de dado (que define o que pode ser armazenado), um nome (como você vai se referir a ela no código) e opcionalmente um valor inicial.

```
// tipo nome valor

int idadeDoUsuario = 25; // declaração completa

double alturaEmMetros = 1.75;

boolean estaChovendo = false;

String nomeCompleto = "Ana Silva";

// Alterando o valor depois:

idadeDoUsuario = 26; // atribuição – sem declarar o tipo de novo
```

Palavras-chave (Keywords)

Palavras-chave são palavras reservadas pela linguagem Java com significado predefinido. Elas não podem ser usadas como nomes de variáveis, classes ou métodos. Java é case-sensitive, então `int` (minúsculo) é uma keyword, mas `Int` (maiúsculo) não é. O IntelliJ e outros IDEs destacam keywords com cor diferente, facilitando sua identificação.

■ **Regra:** Convenção de nomenclatura em Java: use camelCase para variáveis e métodos. A primeira palavra em minúsculo, as demais com inicial maiúscula. Exemplos: `idadeDoCliente`, `calcularMedia`, `isLogado`.

CAPÍTULO 03

Tipos Primitivos Numéricos Inteiros

int, byte, short e long — range, overflow e largura em bits

Os 4 Tipos Inteiros do Java

Java possui 4 tipos primitivos para armazenar números inteiros (sem parte decimal), cada um com uma quantidade diferente de bits e, conseqüentemente, um range diferente de valores. A escolha do tipo correto evita desperdício de memória e problemas de overflow.

Tipo	Bits	Range	Quando usar
byte	8	-128 a 127	Dados binários, protocolo de rede
short	16	-32.768 a 32.767	Raramente usado em Java moderno
int	32	$\pm 2.147.483.647$	■ Padrão para inteiros
long	64	$\pm 9,2 \times 10^{18}$ ■	IDs muito grandes, timestamps

Overflow e Underflow

Overflow ocorre quando um valor ultrapassa o máximo suportado pelo tipo — em vez de lançar um erro, o Java "gira" para o valor mínimo silenciosamente. O mesmo acontece no underflow (abaixo do mínimo). Isso é um comportamento perigoso porque o programa continua rodando com um valor completamente errado. Use `Integer.MAX_VALUE` e `Integer.MIN_VALUE` para verificar os limites.

```
int maxInt = Integer.MAX_VALUE;

System.out.println(maxInt); // 2147483647

System.out.println(maxInt + 1); // -2147483648 (overflow!)

// Sufixo L para literais long maiores que int:

long populacaoMundial = 8_100_000_000L; // L obrigatório!
```

```
// Underscores para legibilidade (Java 7+):  
  
int populacaoBrasil = 215_000_000; // mais legível que 215000000
```

CAPÍTULO 04

Tipos de Ponto Flutuante

float e double — precisão, notação científica e quando usar cada um

float vs double

Números de ponto flutuante representam valores com parte decimal. Java oferece dois tipos: **float** (32 bits, precisão simples) e **double** (64 bits, precisão dupla). O **double** é o tipo padrão para qualquer número decimal em Java — sempre que você escreve um número com ponto decimal como 3.14, o compilador o trata como double.

Para usar **float**, é obrigatório adicionar o sufixo **f** ou **F** ao literal — caso contrário, o compilador tentará tratar o número como double e lançará um erro de tipo. O double é preferível por ser mais preciso e porque as bibliotecas matemáticas do Java (**Math.sqrt**, **Math.pow** etc.) recebem e retornam double.

```
double pi = 3.14159265358979; // double (padrão)

float f = 3.14f; // float — sufixo f obrigatório!

// float x = 3.14; // ■ ERRO: 3.14 é double, não float!

// Notação científica (E-notation):

double pequeno = 1.4E-10; // 1.4 × 10-10 ■

double grande = 3.4E38; // 3.4 × 1038 ■

// Precisão: double é muito mais preciso que float:

float fDiv = 1.0f / 3.0f; // 0.33333334 (imprecisão visível)

double dDiv = 1.0 / 3.0; // 0.3333333333333333 (mais preciso)
```


■ ■ **Atenção:** Para valores monetários, nunca use float ou double — eles introduzem imprecisões de ponto flutuante. Use BigDecimal para cálculos financeiros onde cada centavo importa.

CAPÍTULO 05

char, boolean e String

O 7º e 8º primitivos, Unicode e a classe especial String

O tipo char

O tipo **char** armazena exatamente um caractere — sempre entre aspas simples. Ocupa 2 bytes (16 bits) e é armazenado internamente como um número baseado na tabela Unicode. Isso permite três formas de atribuição: o literal de caractere (mais comum), o valor decimal Unicode, ou o código hexadecimal Unicode.

```
char letra = 'A'; // literal – mais comum

char numero = 68; // decimal: 68 = D na tabela Unicode

char hex = '\u0044'; // hexadecimal Unicode (\u0044 = D)

// Cuidado: char + char resulta em int (soma numérica):

char a = 'A', b = 'B';

System.out.println(a + b); // 131 (65+66) – NÃO concatena!
```

O tipo boolean

O tipo **boolean** armazena apenas dois valores: **true** ou **false**. Por convenção, use prefixos como **is**, **has** ou **can** nos nomes das variáveis boolean — isso torna o código mais legível, como se fizesse uma pergunta lógica.

A classe String

String é uma classe, não um primitivo — mas é tratada de forma especial em Java. Uma String é uma sequência de caracteres imutável: uma vez criada, não pode ser modificada. Operações que parecem modificar uma String na verdade criam uma nova String. Isso garante segurança e permite que o Java reutilize Strings na memória (String Pool).

```
String nome = "Java Masterclass";
```

```
// Concatenação com +:

String msg = "Olá " + "Mundo!"; // "Olá Mundo!"

String info = "Valor: " + 42; // "Valor: 42" (int vira String)


// Imutabilidade – operações criam novas Strings:

String s = "Java";

s = s + " é incrível!"; // cria nova String, não modifica a original
```

CAPÍTULO 06

Operadores e Expressões

Aritméticos, resto, incremento/decremento e operadores compostos

Operadores Aritméticos

Os operadores aritméticos do Java são: **+** (adição ou concatenação), **-** (subtração), ***** (multiplicação), **/** (divisão) e **%** (resto da divisão, também chamado de módulo). O operador **%** é muito útil para verificar se um número é par (**n % 2 == 0**), para criar ciclos em arrays, ou para limitar valores a um intervalo.

Incremento e Decremento

Incrementar ou decrementar uma variável em 1 é tão comum que Java tem operadores especiais para isso. As três formas abaixo produzem o mesmo resultado, mas o operador de pós-incremento (**++**) é o mais compacto e idiomático em Java.

```
int x = 10;

// Três formas de incrementar:

x = x + 1; // forma longa

x += 1; // operador composto

x++; // pós-incremento (mais comum)

// Operadores compostos para todas as operações:

x += 5; // x = x + 5

x -= 3; // x = x - 3

x *= 2; // x = x * 2

x /= 4; // x = x / 4
```

```
x %= 3; // x = x % 3
```

■■ **Atenção:** Os operadores compostos (+=, -=) fazem cast implícito para o tipo da variável à esquerda. Isso pode causar resultados inesperados: `byte b = 10; b -= 5.5;` não dará erro, mas truncará o resultado.

CAPÍTULO 07

Casting e Conversão de Tipos

Widening automático, narrowing com cast explícito e regras de atribuição

Por que precisamos de Casting?

Java não converte automaticamente entre tipos para evitar perda de dados. Quando você faz uma operação com tipos diferentes, o Java "amplia" o resultado para o tipo maior (promoção automática). Por exemplo, somar um byte e um short retorna um int — não um byte. Para atribuir esse resultado de volta a um byte, você precisa de cast explícito.

Widening vs. Narrowing

Widening (alargamento) ocorre automaticamente quando você atribui um tipo menor a um tipo maior — nunca há perda de dados. Narrowing (estreitamento) vai na direção oposta e exige cast explícito, pois pode haver perda de dados. A regra de tamanho é: **byte < short < int < long < float < double**.

```
// WIDENING – automático, sem cast:

int i = 100;

double d = i; // int → double: OK automático (100.0)

// NARROWING – exige cast:

double x = 9.99;

int truncado = (int) x; // double → int: cast obrigatório

System.out.println(truncado); // 9 (NÃO arredonda – apenas corta!)

// Cast em expressões com byte/short:

byte b1 = 10, b2 = 20;

byte soma = (byte)(b1 + b2); // cast necessário – resultado é int
```

■ **Dica:** Cast de double para int trunca a parte decimal — não arredonda. Então `(int) 9.9 == 9`, não 10. Se precisar arredondar, use `Math.round()` antes de fazer o cast.

Parabéns por concluir a apostila!

Você dominou as bases da linguagem Java: tipos primitivos, variáveis, operadores e casting. Esses fundamentos são a base para todas as estruturas que virão — lógica condicional, loops, métodos e orientação a objetos.

Complete Java Masterclass